

Contents

- [FuzzySave Documentation](#)
 - [Welcome to FuzzySave](#)
 - [Documentation Structure](#)
 - [Quick Glance: Code-First vs No-Code](#)
- [01. Architecture & Core Philosophy](#)
 - [Introduction](#)
 - [Package & Assembly Definition \(.asmdef\) Structure](#)
 - [Automated Dependency Installer](#)
 - [Zero-Reflection vs Traditional Reflection](#)
 - [Next Chapter](#)
- [02. Asynchronous Pipeline & Zero-Hitch Architecture](#)
 - [The Core Problem: Why Conventional Systems Stutter](#)
 - [The 3-Phase Pipeline Architecture](#)
 - [Phase Breakdown](#)
 - [Time-Sliced Restoration \(Zero-Hitch Loading\)](#)
 - [Cross-Scene Restoration](#)
 - [Live Performance Profiling Metrics](#)
 - [Next Chapter](#)
- [03. Storage, Security & Data Integrity](#)
 - [Introduction](#)
 - [Fail-Safe Atomic File I/O Pipeline](#)
 - [Cryptography & Anti-Cheat](#)
 - [Byte-Level GZip Compression \(CompressionHandler\)](#)
 - [Serialization Formats](#)
 - [Advanced Type & Reference Converters](#)
 - [Next Chapter](#)
- [04. Visual Save Studio](#)
 - [Overview](#)
 - [1. Dashboard Tab: Visual Mapping & Code Baking](#)
 - [2. Save Explorer Tab: Real-Time Disk File Editor](#)
 - [3. Save Events Tab](#)

- [4. Save Slots Tab](#)
 - [5. Settings Tab](#)
 - [Next Chapter](#)
- [05. Play Mode Live Debugger](#)
 - [Overview](#)
 - [Key Capabilities](#)
 - [Next Chapter](#)
- [06. Tracking Engine, Dynamic Spawns & Graveyard](#)
 - [Overview](#)
 - [1. Static Scene Entities: SaveGuid](#)
 - [2. Dynamic Runtime Spawns: DynamicSpawnTracker](#)
 - [3. The Graveyard Registry \(Destruction Tracking\)](#)
 - [4. Zero-Allocation DTO Architecture](#)
 - [5. CodeRewriter Tool](#)
 - [Next Chapter](#)
- [07. No-Code Runtime Components](#)
 - [Overview](#)
 - [1. AutoSaveManager](#)
 - [2. SaveTriggerZone](#)
 - [3. SaveButtonBinding](#)
 - [4. SaveKeyBinding \(Quick Save / Quick Load\)](#)
 - [5. SaveFeedbackUI](#)
 - [6. SaveActionTrigger](#)
 - [7. FuzzySaveEventManager](#)
 - [Next Chapter](#)
- [08. Slot Metadata & GPU Thumbnails](#)
 - [Overview](#)
 - [1. The SaveSlotMetadata Structure](#)
 - [2. Zero-Stall GPU Thumbnail Capture \(AsyncGPUReadback\)](#)
 - [3. Summary Field Keys](#)
 - [4. Practical Implementation: Building a Save Slot Card](#)
 - [Next Chapter](#)
- [09. Cloud Synchronization](#)

- [Overview](#)
- [Architecture: ICloudStorageProvider](#)
- [Conflict Resolution Policies](#)
- [Supported Cloud Providers](#)
- [Manual Cloud Sync Code Example](#)
- [Next Chapter](#)
- [10. Schema Migrations & Versioning](#)
 - [Overview](#)
 - [Chained Migration Architecture](#)
 - [Registering Migrations](#)
 - [Best Practices for Versioning](#)
 - [Next Chapter](#)
- [11. Attributes & Code-First Development](#)
 - [Overview](#)
 - [Attribute Reference Table](#)
 - [Code-First Example: Player Character & Events](#)
 - [Group-Specific Encryption via Attributes](#)
 - [Seamless Harmony with Visual Save Studio](#)
 - [Next Chapter](#)
- [DTOs & Zero-Allocation Serialization](#)
 - [1. Why DTOs \(Data Transfer Objects\)?](#)
 - [2. The DTO Struct Suite](#)
 - [3. Seamless Implicit Conversions](#)
 - [4. TransformDTO & Physics Controller Safeguards](#)
- [12. Samples & Stress Benchmark Suite](#)
 - [Overview](#)
 - [1. Interactive 3D Demo Scene](#)
 - [2. Performance & Stress Benchmark Arena](#)
 - [Benchmark Stress Testing Controls](#)
 - [The Golden Baseline Audit System](#)
 - [Next Chapter](#)
- [13. Complete C# API Reference](#)
 - [Overview](#)

- [1. In-Memory Data Access](#)
- [2. Asynchronous Save & Load Pipeline](#)
- [3. RAM Snapshots \(Time-Travel\)](#)
- [4. Slot & Companion Metadata](#)
- [5. GPU Thumbnail Screen Capture](#)
- [6. Graveyard & Destruction Tracking](#)
- [7. Diagnostics & Profiling Properties](#)
- [8. Global Events & Delegates](#)

FuzzySave Documentation

Next-Generation, High-Performance, Zero-Hitch, Fail-Safe Save & Load System for Unity

Engineered by **Fuzzy Logic Labs** | Version 1.0.0 | Compatible with Unity 2022.3 LTS & Unity 6+



Welcome to FuzzySave

FuzzySave is an enterprise-grade serialization, persistence, and game state management framework engineered specifically for Unity. It solves the chronic issues found in conventional Unity save systems—such as main-thread lag spikes, corrupted save files, tedious manual boilerplate coding, lack of security, and difficult schema updates.

Whether you are building an indie roguelike requiring rapid RAM snapshots, an open-world RPG managing thousands of dynamic scene entities, or a cross-platform title with cloud synchronization, FuzzySave provides both **no-code visual workflow tools** and **code-first developer APIs**.

Documentation Structure

The documentation is organized into focused, comprehensive chapters. Navigate through the topics below:

#	Chapter	Key Topics Covered
01	Architecture & Philosophy	Design principles, .asmdef modular structure, dependency installer, zero-reflection concept.
02	Asynchronous Pipeline & Zero-Hitch	3-Phase async pipeline, main-thread vs worker thread, time-sliced restoration, cross-scene async loads.
03	Storage, Security & Data Integrity	Atomic file swaps (.tmp → .bak → target), AES-256-CBC, GZip compression, HMAC-SHA256 checksum, custom converters.
04	Visual Save Studio	Full UI Toolkit suite: Dashboard, Drag-and-Drop mapping, Save Explorer (Tree/Raw JSON editor), Events, Slots, Settings, Code Bake.
05	Play Mode Live Debugger	Runtime inspection, live variable injection (cheats/god mode), instant RAM snapshots / time-travel, scene pinging, live graveyard controls.
06	Tracking Engine, Dynamic Spawns & Graveyard	SaveGuid, duplicate detection, DynamicSpawnTracker, runtime prefab re-instantiation, GraveyardRegistry destroy & revive, CodeRewriter.
07	No-Code Runtime Components	AutoSaveManager (rolling slots, pause-aware), SaveTriggerZone (2D/3D checkpoints), SaveButtonBinding, SaveKeyBinding, SaveFeedbackUI, SaveActionTrigger.
08	Slot Metadata & GPU Thumbnails	SaveSlotMetadata, summary field keys (Level, Gold, Health), zero-stall AsyncGPUReadback thumbnail screenshots, save slot UI cards.
09	Cloud Synchronization	CloudSyncManager, conflict resolution policies, UGS Cloud Save, Steam Cloud, PlayFab, Firebase, Custom REST API.
10	Schema Migrations & Versioning	Safe schema evolution, SaveMigrationManager, chained migrations (v1 → v2 → v3), backwards compatibility.

#	Chapter	Key Topics Covered
11	Attributes & Code-First Development	[FuzzySave], [SaveField], [SaveGroup], [SaveEvent], selective persistence without editor tooling.
12	Samples & Stress Benchmark Suite	3D interactive demo scene (DemoScene), 500+ physics item benchmark suite, PerformanceSceneBuilder, Golden Baseline Audit, PerformanceMetricsHUD.
13	Complete C# API Reference	Exhaustive method-by-method reference for FuzzySaveManager, event delegates, signatures, parameters, return values, and code samples.

Quick Glance: Code-First vs No-Code

1. Minimal Code Example

```
using UnityEngine;
using FuzzyLogicLabs.FuzzySave;

public class PlayerProgress : MonoBehaviour
{
    public int currentHealth = 100;
    public int goldCoins = 450;

    // Asynchronous Save
    public async void QuickSaveGame()
    {
        FuzzySaveManager.SetData("player_hp", currentHealth);
        FuzzySaveManager.SetData("player_gold", goldCoins);
        FuzzySaveManager.SetData("player_position", transform.position);

        bool success = await FuzzySaveManager.SaveAsync();
        Debug.Log($"Game saved successfully: {success}");
    }

    // Asynchronous Load
    public async void QuickLoadGame()
    {
        bool success = await FuzzySaveManager.LoadAsync();
        if (success)
        {
            currentHealth = FuzzySaveManager.GetData<int>("player_hp", defaultValue:
100);
            goldCoins = FuzzySaveManager.GetData<int>("player_gold", defaultValue: 0);
            transform.position = FuzzySaveManager.GetData<Vector3>("player_position",
defaultValue: Vector3.zero);
        }
    }
}
```

2. No-Code Visual Workflow

1. Open **Tools > FuzzySave > Visual Save Studio**.

2. Select your component and drag your fields into a **Save Group**.
3. Click **Bake Code** to generate zero-reflection direct accessors.
4. Attach `SaveTriggerZone` to a 3D collider checkpoint in the scene.
5. Hit **Play**—progress is captured without writing a single line of code!

Note

All code examples in this documentation are verified against Unity 2022.3 LTS and Unity 6.

01. Architecture & Core Philosophy

Introduction

In modern game development, persistence systems are frequently implemented as an afterthought—often relying on Unity’s built-in `PlayerPrefs`, simplistic JSON string serialization, or bulky third-party tools that execute all I/O operations directly on the Unity main thread.

FuzzySave was created to eliminate these architectural shortcomings through five core design pillars:

1. **Zero-Hitch Performance:** Offload 99% of serialization, compression, encryption, and disk I/O onto background worker threads (`Task.Run`).
 2. **Fail-Safe Data Integrity:** Never corrupt player save data, even if power is abruptly severed or the game crashes mid-write.
 3. **Zero-Reflection Code Generation:** Generate direct C# getter/setter bakes from visual editor mappings to avoid costly reflection overhead at runtime.
 4. **Dual Workflow (Visual & Code-First):** Give technical designers the ability to bind saves with zero code via UI Toolkit windows and Trigger Zones, while empowering programmers with clean `async/await` C# APIs and attributes.
 5. **Cross-Platform & Cloud Ready:** Native abstraction for local storage alongside industry-standard cloud providers (Unity Gaming Services, Steam Cloud, Microsoft PlayFab, Firebase, and Custom REST APIs).
-

Package & Assembly Definition (.asmdef) Structure

To ensure lightning-fast compilation times, minimal assembly bloat, and strict dependency boundaries, FuzzySave is separated into modular assembly definitions:

```

graph TD
    subgraph Assemblies["FuzzySave Assembly Structure"]
        Installer["FuzzySave.Installer.asmdef<br/>(Lightweight Editor Dependency Resolver)"]
        Runtime["FuzzySave.Runtime.asmdef<br/>(Core Engine, DTOs, Components, Security)"]
        Editor["FuzzySave.Editor.asmdef<br/>(Visual Save Studio, Debugger, CodeGen)"]
        Demo["FuzzySave.Samples.Demo.asmdef<br/>(3D Demo Scene & Sample Controllers)"]
        Benchmark["FuzzySave.Samples.Benchmark.asmdef<br/>(500+ Items Stress Test Suite)"]
        BenchEditor["FuzzySave.Samples.Benchmark.Editor.asmdef<br/>(Auto Scene Builder)"]
    end

    end

    Newtonsoft["com.unity.nuget.newtonsoft-json"]

    Runtime --> Newtonsoft
    Editor --> Runtime
    Editor --> Newtonsoft
    Demo --> Runtime
    Benchmark --> Runtime
    BenchEditor --> Benchmark
    BenchEditor --> Runtime
    BenchEditor --> Editor

```

Assembly Breakdown:

- **FuzzySave.Runtime.asmdef:** Houses the entire runtime foundation: FuzzySaveManager, SaveContainer, SaveEntry, DTO structs, encryption (AES-256), compression (GZip), no-code components (AutoSaveManager, SaveTriggerZone), and tracking registries. It only depends on Unity's official com.unity.nuget.newtonsoft-json.
- **FuzzySave.Editor.asmdef:** Contains all editor-only tools, including VisualSaveStudioWindow (UI Toolkit), PlayModeLiveDebuggerWindow, PartialClassEmitter (code generation), CodeRewriter, custom inspectors, and asset scanners.
- **FuzzySave.Installer.asmdef:** Completely isolated editor bootstrap assembly responsible for detecting package dependencies at editor launch without throwing compilation errors if dependencies are missing.

- **FuzzySave.Samples.Demo.asmdef**: Contains the interactive 3D demo gameplay scripts, UI manager, and sample types.
 - **FuzzySave.Samples.Benchmark.asmdef & Editor**: Houses the high-throughput stress testing arena, physics spawner, and golden baseline verification audit tools.
-

Automated Dependency Installer

FuzzySave depends on the official high-performance `com.unity.nuget.newtonsoft-json` package. To eliminate manual Package Manager configuration, FuzzySave includes a self-healing dependency installer.

How It Works:

1. **Background Inspection ([InitializeOnLoad]):** When the Unity Editor launches or assets are imported, `FuzzySaveDependencyInstaller` scans `manifest.json` and Unity's `PackageCache`.
2. **Non-Intrusive Utility Dialog:** If `com.unity.nuget.newtonsoft-json` is absent, a dedicated modal utility window (`FuzzySaveSetupWindow`) opens automatically.
3. **One-Click Automated Resolution:** Clicking **"Install Newtonsoft Json Automatically"** calls Unity's `UnityEditor.PackageManager.Client.Add("com.unity.nuget.newtonsoft-json@3.2.1")`, displaying an animated progress bar and auto-compiling once complete.

Manual Menu Access:

- Window > Fuzzy Logic Labs > FuzzySave > Setup & Dependencies...
 - Window > Fuzzy Logic Labs > FuzzySave > Install Newtonsoft.Json Dependency
-

Zero-Reflection vs Traditional Reflection

Traditional Unity save tools rely heavily on `System.Reflection` at runtime: inspecting type hierarchies, looping over fields, and calling `FieldInfo.GetValue / SetValue`. On mobile devices and consoles (IL2CPP / AOT), reflection incurs substantial garbage collection (GC) allocation, CPU cache misses, and frame stutters.

FuzzySave offers two high-efficiency alternatives:

1. **Visual Save Studio Code Bake (PartialClassEmitter):** When you map variables in the Visual Save Studio and hit “**Bake Code**”, FuzzySave compiles direct C# getters and setters into `Assets/FuzzyLogicLabs/FuzzySave/Runtime/Generated/FuzzySaveGenerated.cs`. At runtime, data collection executes as direct property lookups at native machine-code speed.
 2. **Zero-Alloc DTO Structs:** Unity math classes (`Vector3`, `Quaternion`, `Color`, etc.) are transformed into lightweight stack-allocated DTO structs with implicit operators (`Vector3DTO dto = transform.position;`).
-

Next Chapter

Proceed to [02. Asynchronous Pipeline & Zero-Hitch](#) to understand how the 3-phase execution pipeline ensures 60/120 FPS performance during high-throughput saving.

02. Asynchronous Pipeline & Zero-Hitch Architecture

The Core Problem: Why Conventional Systems Stutter

Unity's rendering loop is bound to the **Main Thread**. In standard implementations, calling `Save()` performs the following on the main thread:

1. Crawling game objects and reflection reading.
2. Converting data structures into large JSON or binary strings.
3. Compressing with GZip / Deflate.
4. Performing cryptographic hashing and AES encryption.
5. Invoking synchronous file disk I/O (`File.WriteAllText`).

If the save payload is larger than a few kilobytes, the main thread freezes for **20ms to 250ms+**, causing a noticeable dropped frame (lag spike / hitch).

FuzzySave eliminates this with its **3-Phase Asynchronous Pipeline**.

The 3-Phase Pipeline Architecture

sequenceDiagram

autonumber

participant MT as Unity Main Thread

participant WT as Worker Thread (Task.Run)

participant Disk as Disk Storage (.tmp / .bak)

participant Cloud as Cloud Service Provider

Note over MT: Phase 1: State Gathering (0.1ms - 1ms)

MT->>MT: Collect Scene Metadata (Scene Name, Build Index, Ticks)

MT->>MT: Execute FuzzySaveGeneratedBake (Direct C# Accessors)

MT->>MT: Snapshot GlobalGuidRegistry Entities (Transforms -> DTOs)

MT->>MT: Snapshot DynamicSpawnTracker Instances

MT->>MT: Assemble Deep Copy of SaveContainer

Note over WT: Phase 2: Processing & Storage (Background Thread)

MT->>WT: Task.Run(SaveContainerSnapshot)

WT->>WT: Serialize to JSON or Binary Format

WT->>WT: GZip Compression (Optional)

WT->>WT: AES-256-CBC Encryption & PBKDF2 (Optional)

WT->>Disk: Write payload to [Slot].sav.tmp

WT->>Disk: Backup existing target to [Slot].sav.bak

WT->>Disk: Atomic Swap (.tmp -> [Slot].sav)

WT-->>MT: Notify Completion (WorkerDurationMs recorded)

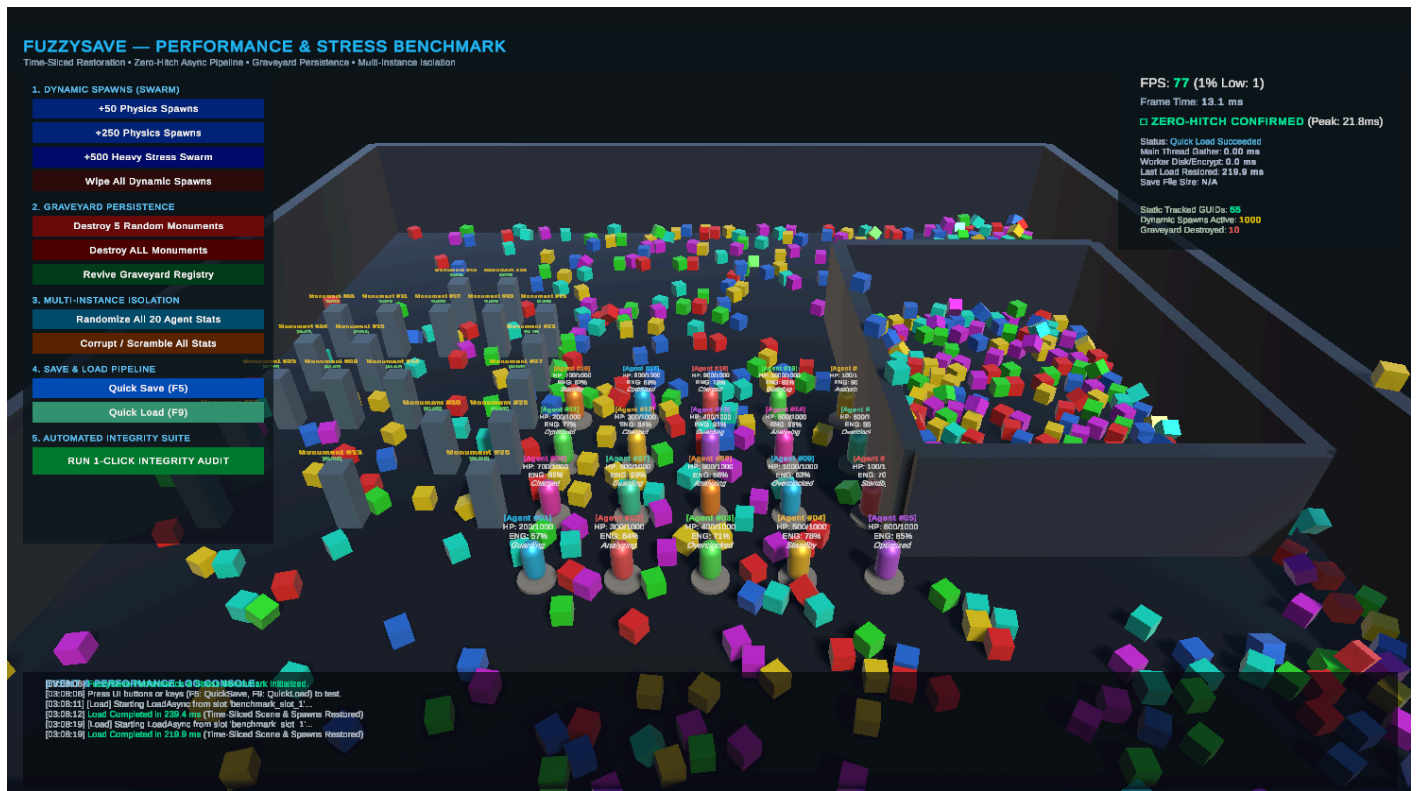
Note over Cloud: Phase 3: Cloud Synchronization & Events

opt Cloud Sync Enabled

MT->>Cloud: UploadSaveAsync(slotName, payload)

end

MT->>MT: Fire OnSaveCompleted(slotName, success)



Phase Breakdown

Phase 1: Main Thread Gathering (~0.1ms – 1.0ms)

Because Unity's C++ native bindings (Transform, GameObject, Component, SceneManager) can only be touched from the main thread, Phase 1 strictly gathers data and maps it into thread-safe Plain Old C# Objects (POCOs) and DTOs:

- Collects active scene name, build index, UTC timestamp, and playtime.
- Direct C# property reads via FuzzySaveGeneratedBake.
- Converts transforms and rigidbodies into stack-allocated DTO structs (Vector3DTO, QuaternionDTO, TransformDTO).
- Clones a deep copy snapshot of SaveContainer.

Phase 2: Background Worker Processing (Task.Run)

All CPU-intensive, allocation-heavy, and disk-bound work runs asynchronously on the .NET thread pool:

- **Serialization:** Full transformation to JSON or Binary without blocking Unity.
- **Compression:** GZip byte stream compression.
- **Encryption:** AES-256-CBC with PBKDF2 cached keys.

- **Atomic I/O:** Flushes to temporary file `.tmp`, validates, creates `.bak` fallback, and atomically swaps file names.

Phase 3: Cloud Sync & Event Dispatch

Once local disk operations conclude:

- The system returns execution safely to Unity's `SynchronizationContext`.
 - If cloud synchronization is configured, `CloudSyncManager` asynchronously uploads payload bytes in the background.
 - Global event `FuzzySaveManager.OnSaveCompleted` fires on the main thread for UI toast feedback.
-

Time-Sliced Restoration (Zero-Hitch Loading)

Loading thousands of objects within a single frame can cause an even worse freeze than saving.

FuzzySave resolves this using an adaptive **Time-Sliced Restoration Engine**:

```

// Excerpt from CoreEngine.cs (LoadAsync)

// 1. Scene GUID Objects: Yield every 500 items to avoid frame hitches
if (s_CurrentContainer.guidObjects != null && s_CurrentContainer.guidObjects.Count > 0)
{
    int guidBatchCounter = 0;
    for (int i = 0; i < s_CurrentContainer.guidObjects.Count; i++)
    {
        var data = s_CurrentContainer.guidObjects[i];
        if (GlobalGuidRegistry.TryGet(data.guid, out var targetComp))
        {
            data.ApplyTo(targetComp);
        }

        guidBatchCounter++;
        if (guidBatchCounter >= 500)
        {
            guidBatchCounter = 0;
            await Task.Yield(); // Hands frame control back to Unity!
        }
    }
}

// 2. Dynamic Prefab Spawns: Yield every 50 instantiations
if (s_CurrentContainer.dynamicSpawns != null && s_CurrentContainer.dynamicSpawns.Count > 0)
{
    int spawnBatchCounter = 0;
    for (int i = 0; i < s_CurrentContainer.dynamicSpawns.Count; i++)
    {
        var spawnData = s_CurrentContainer.dynamicSpawns[i];
        // Instantiate prefab, set GUID, and apply transform & physics...

        spawnBatchCounter++;
        if (spawnBatchCounter >= 50)
        {
            spawnBatchCounter = 0;
            await Task.Yield(); // Prevents GC and engine instantiation stall!
        }
    }
}

```

```
}
```

Cross-Scene Restoration

When loading a save slot created in a different scene, FuzzySave handles scene transitions seamlessly:

flowchart TD

```
Start["LoadAsync(slotName)"] --> CheckScene{"container.sceneName == activeScene.name?"}
CheckScene -- Yes --> RestoreData["Restore GUIDs, Spawns, and Variables"]
CheckScene -- No --> AsyncLoad["SceneManager.LoadSceneAsync(container.sceneName)"]
AsyncLoad --> AwaitScene["await Scene Ready (Task.Yield)"]
AwaitScene --> RestoreData
RestoreData --> Complete["Fire OnLoadCompleted(slotName, true)"]
```

If `container.sceneName != activeScene.name`, FuzzySave:

1. Calls `SceneManager.LoadSceneAsync(container.sceneName)`.
2. Asynchronously waits until the new scene has fully loaded.
3. Automatically maps scene objects and dynamic prefabs to the newly loaded hierarchy.
4. Restores character position, inventory, and health without requiring custom scene transition code.

Live Performance Profiling Metrics

FuzzySave exposes real-time profiling metrics directly on `FuzzySaveManager`:

```
// Inspect execution timings in milliseconds
float mainThreadMs = FuzzySaveManager.LastSaveMainThreadGatherMs;
float workerThreadMs = FuzzySaveManager.LastSaveWorkerDurationMs;

Debug.Log($"Main Thread Gathering: {mainThreadMs:F2} ms | Background Worker I/O: {workerThreadMs:F2} ms");
```

Next Chapter

Proceed to [03. Storage, Security & Data Integrity](#) to explore atomic file operations, AES-256 encryption, GZip compression, and HMAC-SHA256 checksums.

03. Storage, Security & Data Integrity

Introduction

Player save files are vulnerable to two major risks:

1. **File Corruption (Hardware/OS Level):** If the device loses power, battery dies, or the application is forcefully terminated while `FileStream` is actively writing, the save file can be truncated to 0 bytes or left in a corrupt, unreadable state.
2. **Cheating & Tampering (User Level):** Plain JSON/XML save files stored in `Application.persistentDataPath` can easily be opened in a text editor like Notepad, allowing players to manipulate currency, health, or unlocked levels.

FuzzySave incorporates industrial-grade storage safeguards, cryptography, and compression routines to solve both problems.

Fail-Safe Atomic File I/O Pipeline

FuzzySave never writes directly to the primary save file. Instead, it utilizes a 3-step **Atomic Staged Write** protocol:

```
flowchart TD
    Payload["Memory Payload (Encrypted/Compressed)"] --> Step1["Step 1: Write to [Slot].sav.tmp<br/>(Full FlushAsync & Close)"]
    Step1 --> Verify{"Write Valid & Complete?"}
    Verify -- Yes --> Step2["Step 2: Copy Existing [Slot].sav to [Slot].sav.bak"]
    Verify -- No (Crash) --> Abort["Target File Intact (No Data Loss)"]
    Step2 --> Step3["Step 3: Atomic Swap<br/>([Slot].sav.tmp -> [Slot].sav)"]
    Step3 --> Success["Save Completed Safely"]
```

Safety Guarantees:

- **Crash During Step 1:** The original `[Slot].sav` file remains untouched. The incomplete `.tmp` file is safely discarded on next launch.
- **Crash During Step 2 or 3:** The `.bak` backup file guarantees that full player progress can be seamlessly recovered without loss.
- **Result: Zero possibility of 0-byte corrupt save files.**

Cryptography & Anti-Cheat

1. AES-256-CBC Encryption (EncryptionHandler)

- **Algorithm:** Advanced Encryption Standard (AES) with a 256-bit key in Cipher Block Chaining (CBC) mode.
- **Key Derivation:** Cryptographically robust PBKDF2 key derivation using Rfc2898DeriveBytes (SHA-256 hash algorithm, 1000 iterations).
- **High-Performance Memory Cache:** PBKDF2 is computationally heavy by design to prevent brute-force attacks. To ensure this does not introduce runtime overhead during repeated saves, FuzzySave caches derived keys and initialization vectors (IVs) inside a thread-safe `ConcurrentDictionary<string, byte[]>`:

```
// Excerpt from SecurityHandlers.cs
private static readonly ConcurrentDictionary<string, byte[]> s_KeyCache =
    new ConcurrentDictionary<string, byte[]>();

public static byte[] DeriveKey(string password, byte[] salt)
{
    string cacheKey = password + Convert.ToBase64String(salt);
    return s_KeyCache.GetOrAdd(cacheKey, _ =>
    {
        using var pbkdf2 = new Rfc2898DeriveBytes(password, salt, 1000,
HashAlgorithmName.SHA256);
        return pbkdf2.GetBytes(32); // 256 bits
    });
}
```

2. HMAC-SHA256 Tamper Proofing (ChecksumHandler)

- Whenever a save file is written, an HMAC-SHA256 signature is calculated over the entire encrypted payload using the developer's private security key.
 - Upon loading, the checksum is computed and compared before deserialization begins.
 - If a player modifies even a single byte in a hex editor, the checksum fails, preventing corrupt data or cheated values from penetrating the game's memory.
-

Byte-Level GZip Compression (CompressionHandler)

Uncompressed JSON files in large games can easily reach **10MB to 50MB+**, consuming substantial disk space and generating high network payloads during cloud sync.

- FuzzySave utilizes `System.IO.Compression.GZipStream` with `CompressionLevel.Optimal`.
- Compression occurs on the background thread immediately after serialization and before AES encryption.
- **Size Reduction:** Typical game state JSON payloads achieve between **70% and 90% size reduction** (e.g., a 4.2 MB raw world state compresses down to ~420 KB).

Serialization Formats

FuzzySave supports multiple serialization formats configured via `FuzzySaveSettings.defaultFormat`:

Serialization Engine	File Extension	Best Used For
JsonSaveSerializer	.sav	Human-readable inspection in Save Explorer, rapid debugging, cross-engine interoperability.
BinarySaveSerializer	.bin	Maximum compactness, lowest serialization latency, consoles and memory-constrained mobile devices.

Advanced Type & Reference Converters

Standard serializers fail when serializing Unity-specific types (such as `GameObject`, `Component`, or `ScriptableObject`) due to circular references and missing parameterless constructors. FuzzySave implements custom JSON converters:

1. GameObjectReferenceConverter

- Instead of serializing the entire scene graph, it extracts the target object’s `SaveGuid` or deterministic hierarchy path (`Scene/Parent/Name#SiblingIndex`).

- Upon deserialization, it queries `GlobalGuidRegistry` or scene hierarchy to reconnect the live reference automatically.

2. ScriptableObjectReferenceConverter

- Serializes the `ScriptableObject`'s identity (name and relative path).
 - Deserializes by resolving instances from runtime cache or `Resources.Load<ScriptableObject>()`.
-

Next Chapter

Proceed to [04. Visual Save Studio](#) to tour the UI Toolkit editor suite, drag-and-drop mapping, and the live Save Explorer.

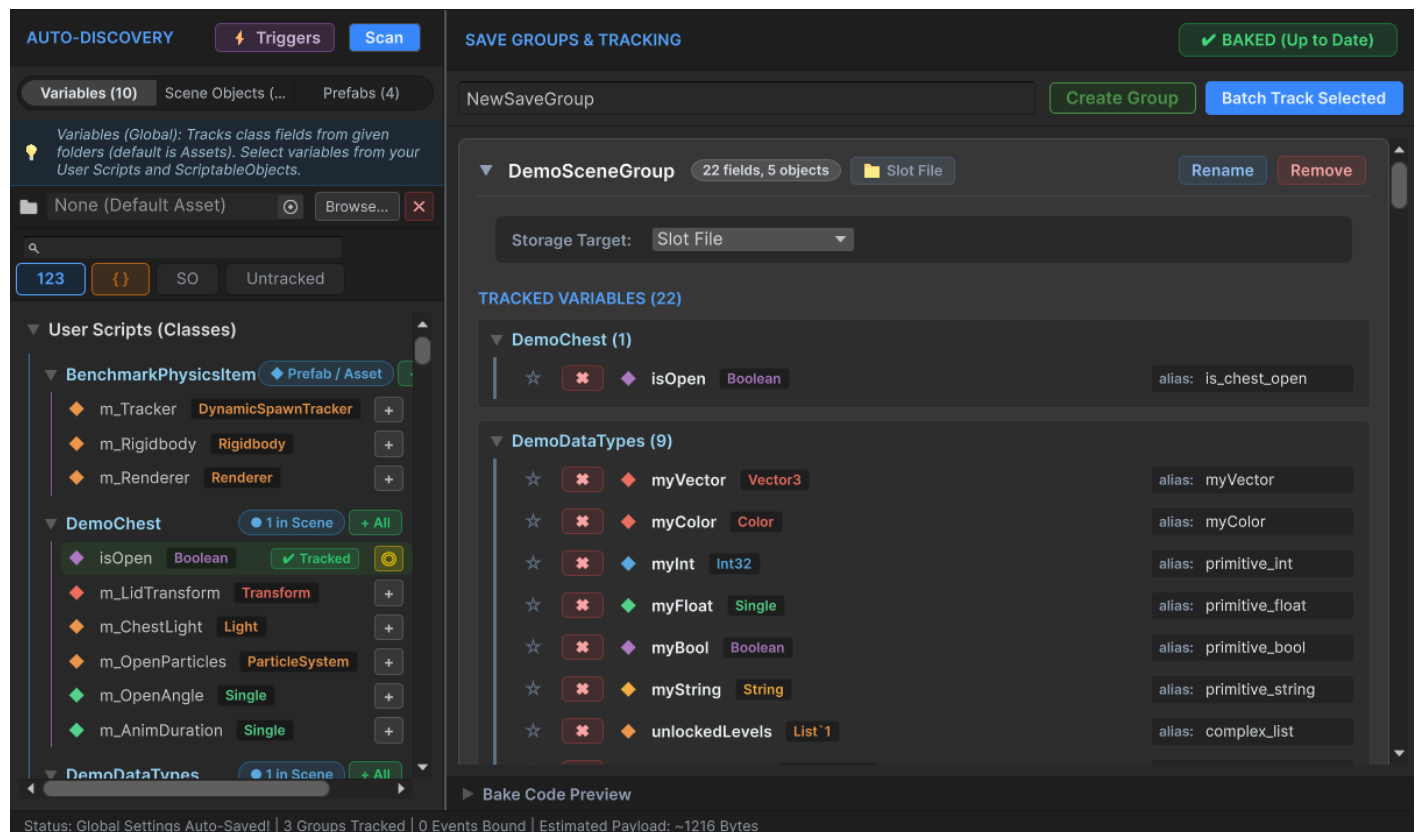
04. Visual Save Studio

Overview

Visual Save Studio is FuzzySave's flagship editor environment built entirely with Unity's modern **UI Toolkit** and **UI Builder**. It provides a comprehensive, visual command center to configure persistence, inspect live disk files, bind event triggers, and generate high-performance C# code without writing boilerplate.

Open the studio via the Unity menu: **Window > Fuzzy Logic Labs > FuzzySave > Visual Save Studio**

1. Dashboard Tab: Visual Mapping & Code Baking



The **Dashboard** tab is the heart of no-code configuration:

Left Panel: Component & Variable Discovery

- **3 Category Modes:**
 - **User Scripts:** Scans all custom MonoBehaviour and C# scripts in the project.

- **Engine Components:** Scans Unity built-in components (Transform, Rigidbody, Camera, Light, etc.).
- **Save Map:** Provides an aggregated list showing which fields are mapped to which save groups.
- **Target Folder Filter:** In large projects with thousands of third-party assets, use the folder picker to restrict scanning strictly to your team's code folder (e.g., Assets/MyGame/Scripts).
- **Data Type Filters:** Toggle visibility for Primitives, Events, Custom Classes, ScriptableObjects, and Untracked Only.

Center Panel: Save Groups (Containers)

- Group your data logically (e.g., PlayerGroup, WorldStateGroup, SettingsGroup).
- **Drag-and-Drop:** Simply grab any variable or property from the left panel and drop it onto a Save Group card.
- **Aliases:** Assign clean, space-saving JSON keys to fields (e.g., in-code `m_CurrentPlayerHealthPoints` becomes "hp" in the save file).
- **Summary Variable Toggle (`isSummaryVariable`):** Flagging a field (like level or gold) ensures it is extracted into `SaveSlotMetadata.summaryValues` for display on save slot UI cards without loading the full save.

Right Panel: Group Inspector & Live C# Preview

Configure per-group settings:

- **Storage Target:**
 - `SlotFile`: Bundled into the active slot file (e.g., `slot_1.sav`).
 - `GlobalFile`: Shared global file across all slots (e.g., `global_settings.sav` for audio volume and graphics).
 - `CustomFile`: Saved to a dedicated file pattern.
- **Trigger Policy:** `ManualOnly`, `OnSceneChange`, `OnAppPause`, `IntervalTimer`, `OnEventFired`.
- **Encryption & Compression Overrides:** Choose whether this specific group overrides global security rules.
- **Live C# Preview:** Shows the exact C# code that will be compiled.

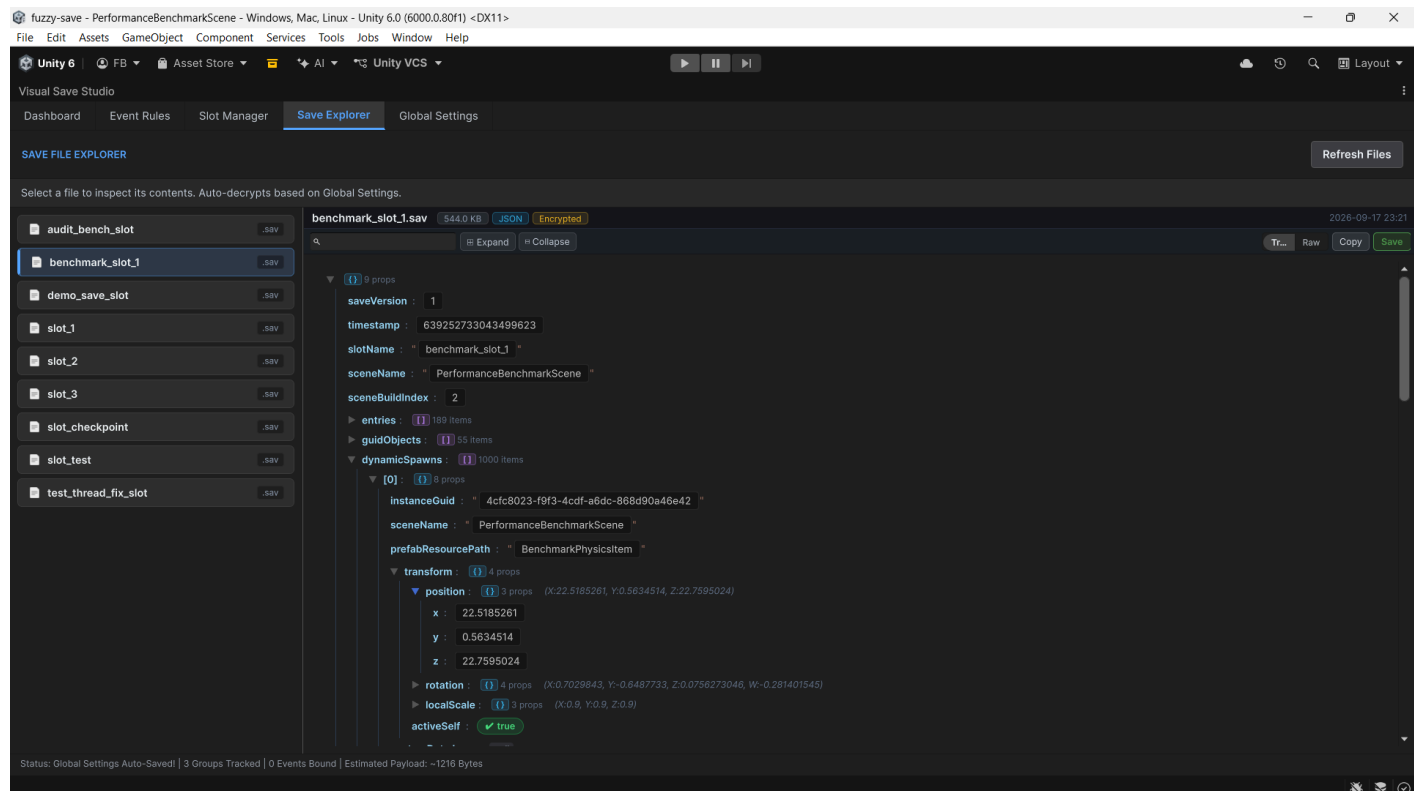
Zero-Reflection Code Baking (PartialClassEmitter)

Clicking the green **“Bake Code”** button generates direct getter/setter code in:

Assets/FuzzyLogicLabs/FuzzySave/Runtime/Generated/FuzzySaveGenerated.cs

This eliminates reflection entirely at runtime, delivering native C# speed.

2. Save Explorer Tab: Real-Time Disk File Editor



The **Save Explorer** allows developers and QA testers to inspect, search, and edit disk save files directly inside Unity without needing an external text editor.

Features:

- **Dual Viewing Modes:**
 - **Tree Mode:** Interactive visual hierarchy with { } (n props) and [] (n items) badges.
 - **Raw Mode:** Direct formatted JSON text editing.
- **JetBrains Mono Typography:** Integrated custom monospace font for readability.
- **Visual Inspectors:**
 - **Color Swatch Preview:** Properties with r, g, b, a values render an actual interactive color swatch.

- **Single-Line Vector Preview:** Vector objects render concisely as (X: 1.0, Y: 2.0, Z: 3.0).
 - **Inline Boolean Toggles:** Clickable pill buttons to flip true/false instantly.
 - **Inline Numeric/String Inputs:** Edit values directly in the tree.
 - **Live Search Filtering:** Typing in the search bar dynamically filters keys and values, automatically expanding parent branches.
 - **Save File Button:** Writes modifications back to disk using the same atomic, fail-safe pipeline as runtime saves.
 - **File Metadata Card:** Displays file size, format, encryption status, compression ratio, and last modified UTC timestamp.
-

3. Save Events Tab

Configure automated save triggers without writing code:

- **Built-in Triggers:** `OnSceneLoaded`, `OnSceneUnloaded`, `OnApplicationPause`, `OnApplicationQuit`.
 - **Interval Timers:** Set periodic triggers (e.g., save `AutosaveGroup` every 180 seconds).
 - **Custom Event Rules:** Bind named game events (e.g., `"OnBossDefeated"`, `"OnQuestCompleted"`) to specific save groups.
-

4. Save Slots Tab

Manage physical save files stored on the local machine:

- Displays all detected `.sav` and `.bin` files under `FuzzySaveData`.
 - Shows formatted file size, creation date, and last modified timestamp.
 - Individual **Delete Slot** buttons and global **Clear All Slots** command.
-

5. Settings Tab

Visual editor for `FuzzySaveSettings.asset`:

- Configure save directory name (default: `FuzzySaveData`).

- Toggle default serialization format (Json or Binary).
 - Configure global AES-256 password and GZip compression.
 - Enable Graveyard tracking and default destruction action.
 - Setup Cloud Synchronization providers and endpoints.
-

Next Chapter

Proceed to [05. Play Mode Live Debugger](#) to learn how to inspect memory, execute live cheats, and manipulate RAM snapshots while the game is running.

05. Play Mode Live Debugger Overview

The **Play Mode Live Debugger** is a real-time inspection, diagnostic, and cheat console that provides an X-ray view into FuzzySave's runtime memory state while your game is actively playing.

Open the debugger via the Unity menu: **Tools > FuzzySave > Play Mode Live Debugger**

FUZZY SAVE - PLAY MODE LIVE DEBUGGER

LIVE (PLAY MODE ACTIVE)

Auto-Refresh

Refresh

▼ Live Memory State (189 Variables)

Filter variables by key or type...

(189 Variables)

benchmark_timestamp

System.String

2026-09-17T20:21:44.3499623Z

Inject

agent_id_d8a4612c-6c45-4781-8d4f-6b996d3714fa

System.Int32

20

Inject

agent_id_757864d6-5dc8-4f6f-87d1-6d6215f71799

System.Int32

18

Inject

agent_id_e11eb1dd-3fb3-4ac6-8dc6-87ad7ea16e79

System.Int32

14

Inject

agent_id_044ce131-e9db-49aa-b466-5cd168012df5

System.Int32

8

Inject

agent_id_422d94c6-9d69-4921-856a-7cbdccb052a8

System.Int32

▶ Registered Scene GUID Objects (55 Objects)

▶ Active Dynamic Spawns (1000 Spawns)

▶ Graveyard (10 Destroyed)

Status: Live Debugger Active

Key Capabilities

1. Real-Time Status & Slot Controls

- **Live Status Badge:** Displays green ● LIVE (ONLINE) while in Play Mode. In Edit Mode, a clear warning card explains that memory inspection requires an active game session.
- **Instant Slot Operations:** Select any slot from the dropdown or create a new slot name, then trigger **Save**, **Load**, or **Delete** with a single click.

2. RAM Snapshots & Instant Time-Travel

RAM Snapshots allow developers and QA testers to capture the exact game state into memory without touching the hard drive:

- **Take Snapshot:** Saves the entire state into a RAM dictionary keyed by a custom ID (e.g., "before_boss").
- **Restore Snapshot:** Restores all player variables, scene GUID positions, and spawned entities within < **1 millisecond**.
- **Ideal For:** Fast testing loops, trial-and-error combat balancing, and debugging transient bugs.

```
// Programmatic equivalent of the snapshot buttons:  
FuzzySaveManager.TakeSnapshot("boss_room");  
  
// Restore immediately on defeat:  
FuzzySaveManager.RestoreSnapshot("boss_room");
```

3. Live Variable Injection (God Mode & Cheating)

The **In-Memory Variables** panel lists every variable registered in s_CurrentContainer:

- **Real-Time Polling:** Values update automatically every 500ms as gameplay progresses.
- **Live Mutate & Inject:**
 1. Type a new value into any variable field (e.g., change player_hp from 100 to 9999).
 2. Click **Inject** (or press Enter).
 3. FuzzySave immediately writes the value back to the live GameObject in the scene.

- **Search Filtering:** Quick search bar to locate specific stats in complex games with hundreds of variables.
-

4. Scene Object & Spawn Tracking with Ping

- **Tracked Scene GUIDs:** Lists every static scene object monitored by a SaveGuid component along with its current transform position.
 - **Active Dynamic Spawns:** Lists every runtime-instantiated entity tracked by DynamicSpawnTracker.
 - **[Ping] Button:** Clicking **Ping** instantly highlights and selects the GameObject in Unity's Hierarchy and focuses it in the Scene View!
-

5. Live Graveyard (Destruction) Management

When objects registered with SaveGuid are destroyed in the scene (such as opened chests, collected coins, or defeated enemies), they enter the **Graveyard Registry**:

- The Graveyard panel displays all currently dead GUIDs.
 - **[Revive] Button:** Removes the object from the graveyard, allowing it to respawn when the scene or slot is reloaded.
 - **[Clear Graveyard]:** Resets the destruction history across the current session.
-

Next Chapter

Proceed to [06. Tracking Engine, Dynamic Spawns & Graveyard](#) to explore the mechanics behind SaveGuid, runtime entity instantiation, and destruction tracking.

06. Tracking Engine, Dynamic Spawns & Graveyard

Overview

Saving a game involves more than just storing isolated variables; it requires preserving the spatial state and existence of game entities:

1. **Static Scene Objects:** Objects placed in the scene at build time (e.g., doors, chests, levers).
2. **Dynamic Runtime Entities:** Objects spawned dynamically via `Instantiate` (e.g., dropped loot, spawned enemies).
3. **Destroyed Entities (The Graveyard):** Scene objects that were killed, consumed, or destroyed and must **not** respawn when reloading.

FuzzySave handles all three scenarios with zero runtime boilerplate.

1. Static Scene Entities: SaveGuid

The `SaveGuid` component should be attached to any `GameObject` in your scene hierarchy whose transform, physics, or active state must be persisted.

```
using UnityEngine;
using FuzzyLogicLabs.FuzzySave;

[RequireComponent(typeof(SaveGuid))]
public class TreasureChest : MonoBehaviour
{
    // SaveGuid handles position, rotation, and active state automatically!
}
```

Key Features:

- **Automatic GUID Generation:** Upon being added to a `GameObject`, `SaveGuid` generates a unique string identifier (`System.Guid.NewGuid()`).
- **Duplicate Detection:** If a designer duplicates a `GameObject` or pastes a prefab in the scene, `SaveGuid.OnValidate` detects identical IDs across the scene and automatically generates a fresh GUID.

- **Save Flags (SaveFlags):** Granularly toggle what data to capture:
 - Position: World coordinates.
 - Rotation: World rotation quaternion.
 - Scale: Local transform scale.
 - ActiveState: `gameObject.activeSelf`.
 - RigidbodyState: Velocity and angular velocity.
 - CustomState: Custom serialized JSON payload.
 - All: Enables all flags.
- **Position Threshold (`m_PositionThreshold`):** Skips writing delta transform updates if the object moved less than the threshold (e.g., `0.001f`), saving memory and CPU cycles.
- **Unity 6 Compatibility:** Automatically adapts to Unity 6's updated physics APIs using `#if UNITY_6000_0_OR_NEWER` (`rb.linearVelocity` vs `rb.velocity`).

2. Dynamic Runtime Spawns: `DynamicSpawnTracker`

Objects spawned at runtime (such as projectiles, floating crystals, or summoned monsters) do not exist in the scene hierarchy when the scene starts.

To make them persistent:

1. Attach `DynamicSpawnTracker` to your prefab.
2. Provide the prefab lookup path via:
 - **Resources Folder:** Relative path in `m_PrefabResourcePath` (e.g., `"Prefabs/LootCrystal"`).
 - **Settings Registry:** Direct asset reference in `FuzzySaveSettings.registeredPrefabs`.

```
Player spawns loot → DynamicSpawnTracker assigns Instance GUID
                    |
                    ▼ (Game Saved)
    Prefab Path + Position + Velocity saved to SaveContainer
                    |
                    ▼ (Game Reloaded)
    Old runtime instances destroyed → Fresh prefabs instantiated
    at saved positions with identical velocities and IDs
```

3. The Graveyard Registry (Destruction Tracking)

The Problem:

If a player opens and destroys a treasure chest in the scene and then saves, reloading the scene will cause Unity to instantiate the default scene hierarchy—bringing the destroyed chest back from the dead!

The FuzzySave Solution:

FuzzySave includes an automated **Graveyard Tracking Registry**:

- Whenever an object with a SaveGuid is destroyed, its GUID is registered in GraveyardRegistry.
- Destructions are captured automatically via `SaveGuid.OnDestroy()` or explicitly via `FuzzySaveManager.Destroy(gameObject)`.
- When loading a save slot, FuzzySave inspects the `destroyedGuids` list and automatically purges or deactivates those entities before the player sees them.

```
// Recommended method to destroy persistent objects:
FuzzySaveManager.Destroy(enemyGameObject);

// Check if an entity is recorded as destroyed:
bool isDead = FuzzySaveManager.IsDestroyed("chest_room_3_guid");

// Revive an entity (e.g., respawning a boss or resetting a dungeon):
FuzzySaveManager.Revive("chest_room_3_guid");

// Clear all destruction records:
FuzzySaveManager.ClearGraveyard();
```

Graveyard Actions:

Configurable via `SaveGuid.GraveyardAction` or `FuzzySaveSettings.defaultGraveyardAction`:

- **Destroy:** Completely removes the `GameObject` from memory (`UnityEngine.Object.Destroy`).
 - **Deactivate:** Keeps the `GameObject` intact but sets `gameObject.SetActive(false)`.
-

4. Zero-Allocation DTO Architecture

Serializing native Unity classes directly creates reference loops and heavy GC garbage. FuzzySave uses stack-allocated DTO (Data Transfer Object) structs:

Unity Type	FuzzySave DTO	Conversion Syntax
Vector2, Vector3, Vector4	Vector2DTO, Vector3DTO, Vector4DTO	Vector3DTO dto = transform.position; (Implicit)
Quaternion	QuaternionDTO	QuaternionDTO rot = transform.rotation; (Implicit)
Color, Color32	ColorDTO, Color32DTO	ColorDTO c = myColor; (Implicit)
Rect, Bounds	RectDTO, BoundsDTO	RectDTO r = myRect; (Implicit)
Transform + Physics	TransformDTO	Encapsulates pos, rot, scale, active, and velocities

CharacterController & NavMeshAgent Safety

Applying positions directly to a Transform while a CharacterController or NavMeshAgent is active can cause physics stutter or immediate position snapbacks. TransformDTO.ApplyTo handles this safely:

```
// Excerpt from DTOs.cs (TransformDTO.ApplyTo)
var cc = go.GetComponent<CharacterController>();
var agent = go.GetComponent<UnityEngine.AI.NavMeshAgent>();

// Temporarily disable controllers before applying position
if (cc != null) cc.enabled = false;
if (agent != null) agent.enabled = false;

transform.position = position;
transform.rotation = rotation;

if (cc != null) cc.enabled = true;
if (agent != null) agent.enabled = true;
```

5. CodeRewriter Tool

Found in `Editor/CodeMod/CodeRewriter.cs`, the `CodeRewriter` is an editor utility that uses regular expressions to convert standard Unity math types in your C# scripts to `FuzzySave` DTOs (and vice-versa) with a single click, ensuring compile-time type safety for custom serialization pipelines.

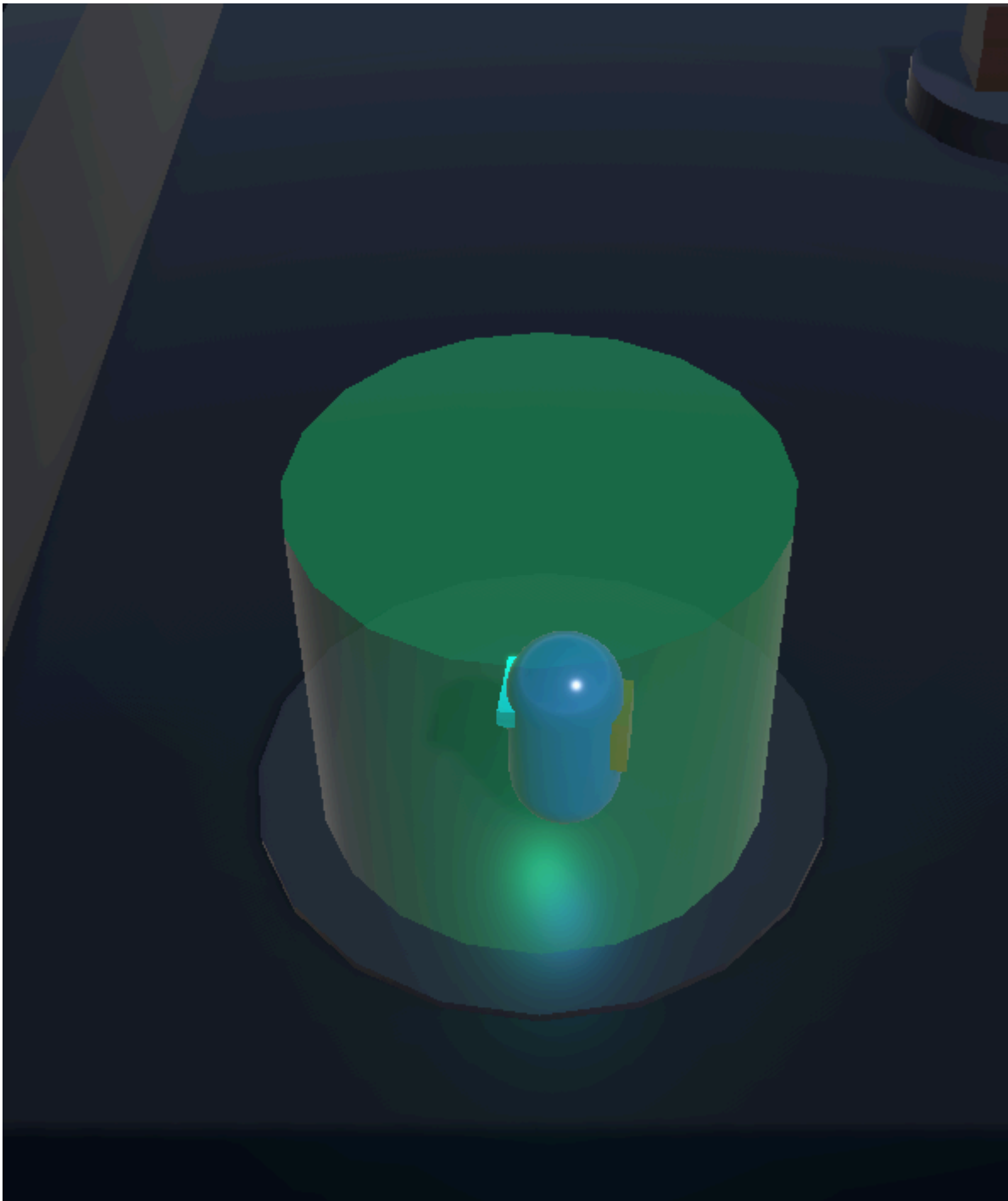
Next Chapter

Proceed to [07. No-Code Runtime Components](#) to explore the ready-to-use checkpoint zones, rolling autosave managers, and UI bindings.

07. No-Code Runtime Components

Overview

FuzzySave includes a rich suite of plug-and-play MonoBehaviour components designed for level designers, technical artists, and teams seeking zero-code gameplay persistence. Simply attach these components to scene objects or UI prefabs to establish full save and load mechanics.



1. AutoSaveManager

The AutoSaveManager runs autonomously in the background to handle periodic saves and lifecycle triggers.

```
Inspector: [FuzzySave/Auto Save Manager]
├─ Periodic Auto-Save:
│   ├─ Enable Interval Save: [x]
│   ├─ Interval Seconds: 300 (5 minutes)
│   └─ Pause When TimeScale Zero: [x] (Skips saving while in Pause Menu)
├─ Slot Management:
│   ├─ Use Rolling Slots: [x]
│   ├─ Rolling Slot Prefix: "autosave_"
│   └─ Rolling Slot Count: 3 (Cycles: autosave_1 -> autosave_2 -> autosave_3)
└─ Lifecycle Triggers:
    ├─ Save On Scene Loaded: [ ]
    ├─ Save On Application Pause: [x]
    └─ Save On Application Quit: [x]
```

Key Highlights:

- **Rolling Slots Protection:** Instead of repeatedly overwriting a single file, the manager cycles through autosave_1, autosave_2, and autosave_3. If a player saves right before dying, previous checkpoints remain intact.
- **TimeScale Zero Protection:** When players open inventory or pause screens (`Time.timeScale == 0`), auto-saving is halted to avoid capturing broken game states.

2. SaveTriggerZone

Attach SaveTriggerZone to any GameObject with a 2D or 3D Collider (`isTrigger = true`) to create checkpoints, level exit saves, or auto-load gates.

Features:

- **Collider Support:** Works with 3D (BoxCollider, SphereCollider) and 2D (BoxCollider2D, CircleCollider2D).
- **Filtering:** Filter by TargetTag (default: "Player") and LayerMask.

- **Supported Actions:** Save, Load, SaveGroup, LoadGroup, DeleteSlot, ClearAllSlots.
 - **Cooldown & One-Shot:**
 - `m_CooldownSeconds`: Prevents double triggers when a player paces inside the zone.
 - `m_OneShot`: Ensures the checkpoint only activates once in its lifetime.
 - **Scene View Gizmos:** Automatically draws colored 3D wire/solid gizmos in the Scene View:
 - **Green:** Save Action
 - **Blue:** Load Action
 - **Red:** Delete Action
-

3. SaveButtonBinding

Attach `SaveButtonBinding` to any standard Unity UI Button to convert it into a save or load button without writing click listeners.

Anti-Spam Lockout:

When saving or loading begins, `SaveButtonBinding` automatically sets `button.interactable = false`. Once the asynchronous operation completes, it re-enables the button, preventing players from spamming file writes.

4. SaveKeyBinding (Quick Save / Quick Load)

Provides PC keybinding support (Default: **F5** for Quick Save, **F9** for Quick Load):

- Internal cooldown protection (1.0s) to prevent rapid key hammering.
 - Optional audio clips for successful save (`m_SaveSound`) and load (`m_LoadSound`).
-

5. SaveFeedbackUI

A completely autonomous visual notification component. Drop it onto any Canvas UI element or toast banner prefab:

- **Zero Configuration:** Automatically registers itself to global events on `FuzzySaveManager`.
- **Smooth Fading:** Controls a `CanvasGroup` with customizable fade-in and fade-out animations.

- **Spinning Indicator:** Rotates an icon transform (m_SpinnerIcon) during active I/O.
 - **State Messages:** Shows "Saving...", "Game Saved", "Loading...", or "Operation Failed!".
-

6. SaveActionTrigger

A versatile bridge component exposing parameterless and parameterized public methods ready for connection with:

- **UnityEvents** (Button.onClick, UnityEvent)
- **Animation Events** (save game at the end of a cutscene animation)
- **Timeline Signals**
- **Dialogue Systems** (save progress when finishing a quest dialogue)

```
// Exposed public methods:
public void Save();
public void Load();
public void SaveActiveSlot();
public void LoadActiveSlot();
public void SaveSlot(string slotName);
public void LoadSlot(string slotName);
public void SaveGroup(string groupName);
public void LoadGroup(string groupName);
public void DeleteSlot(string slotName);
public void ClearAllSlots();
public void TriggerCustomEvent(string eventName);
```

7. FuzzySaveEventManager

Automatically instantiated upon scene load via [RuntimeInitializeOnLoadMethod], this persistent DontDestroyOnLoad runner monitors global event rules defined in FuzzySaveSettings.eventRules.

Next Chapter

Proceed to [08. Slot Metadata & GPU Thumbnails](#) to explore rich slot data cards and zero-stall GPU screenshot capture.

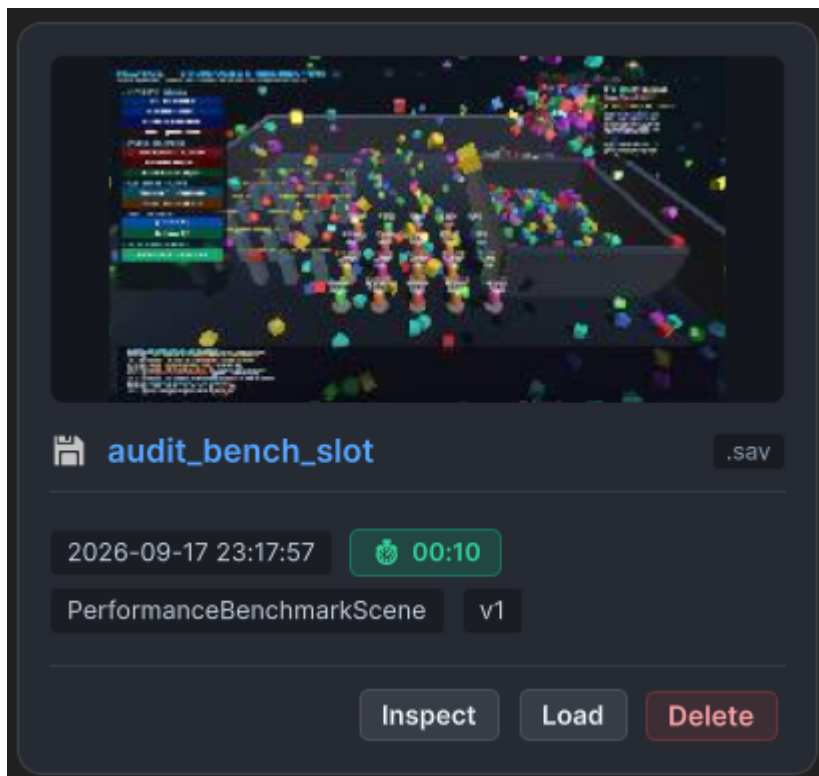
08. Slot Metadata & GPU Thumbnails

Overview

Modern games rarely present players with a raw text list of save files. Players expect rich **Save Slot Cards** showing:

- A visual screenshot of where they saved.
- Total playtime formatted as 04h 23m 12s.
- Last modified timestamp in their local timezone.
- Key gameplay stats (e.g., Level, Gold, Difficulty) without having to load the actual save data.

FuzzySave provides this out of the box through **Lightweight Companion Metadata** and **Zero-Stall GPU Thumbnail Capture**.



1. The SaveSlotMetadata Structure

Whenever a slot is saved, FuzzySave writes a lightweight companion metadata file (e.g., slot_1.meta alongside slot_1.sav).

```
[Serializable]
public class SaveSlotMetadata
{
    public string slotName;
    public string displayName;
    public bool exists;
    public long timestampUtcTicks;
    public float playtimeSeconds;
    public string sceneName;
    public int sceneBuildIndex;
    public int saveVersion;
    public long sizeBytes;
    public string thumbnailBase64; // High-efficiency JPEG Base64
    public Dictionary<string, string> summaryValues; // Fast summary stats
}
```

Formatting Utilities:

```
string dateStr = metadata.GetFormattedDate();      // "2026-09-17 19:45:00"
string timeStr = metadata.GetFormattedTimeOnly(); // "19:45:00"
string sizeStr = metadata.GetFormattedSize();      // "428.5 KB"
string playStr = metadata.GetFormattedPlaytime(); // "02h 15m 43s"
```

Fast Slot Querying:

Reading 10 large save files just to render a load menu can cause significant disk thrashing. Instead, query the companion metadata files in less than a millisecond:

```
// Fetches all slot metadata in parallel without loading main save payloads!
List<SaveSlotMetadata> allSlots = await FuzzySaveManager.GetAllSlotMetadataAsync();
```

2. Zero-Stall GPU Thumbnail Capture (AsyncGPUReadback)

The Problem with Texture2D.ReadPixels:

Standard Unity screenshot solutions call `ReadPixels`, which halts the CPU main thread until the GPU finishes rendering the current frame buffer—causing a noticeable 15ms to 40ms hitch.

The FuzzySave Solution:

FuzzySave uses Unity's native `AsyncGPUReadback` API:

1. Takes a copy of the screen buffer or active camera `RenderTarget`.
2. Dispatches an asynchronous GPU readback request.
3. Once the GPU completes rendering, bytes are passed to a background worker to encode into a compressed JPEG Base64 string.
4. **Zero rendering stalls, zero dropped frames.**

Configuration in FuzzySaveSettings:

- **thumbnailCaptureMode:** `AsyncGPUReadback` (Default), `Synchronous`, `Disabled`.
- **thumbnailWidth:** Default 240 px.
- **thumbnailHeight:** Default 135 px (16:9 ratio).
- **thumbnailQuality:** Default 70 (balanced compression).

```
// Manual capture API:  
string base64Image = await FuzzySaveManager.CaptureThumbnailBase64Async(width: 240,  
height: 135, quality: 70);
```

3. Summary Field Keys

Instead of loading an entire 5MB world save just to display the player's level and gold on the menu, mark your variables as summary keys:

1. In `FuzzySaveSettings`, add keys to `slotSummaryFieldKeys` (e.g., `"level"`, `"player_gold"`, `"difficulty"`).

2. Or in Visual Save Studio, toggle the **Summary** checkmark on any mapped field.

3. Programmatic lookup:

```
// Inspect summary values without loading the slot:  
SaveSlotMetadata meta = await FuzzySaveManager.GetSlotMetadataAsync("slot_1");  
  
if (meta.summaryValues.TryGetValue("player_gold", out string goldStr))  
{  
    goldText.text = $"Gold: {goldStr}";  
}
```

4. Practical Implementation: Building a Save Slot

Card

```
using UnityEngine;
using UnityEngine.UI;
using FuzzyLogicLabs.FuzzySave;
using System;

public class SaveSlotCardUI : MonoBehaviour
{
    [SerializeField] private RawImage m_ThumbnailImage;
    [SerializeField] private Text m_TitleText;
    [SerializeField] private Text m_PlaytimeText;
    [SerializeField] private Text m_DateText;
    [SerializeField] private Text m_SummaryText;

    public async void PopulateCard(string slotName)
    {
        SaveSlotMetadata meta = await FuzzySaveManager.GetSlotMetadataAsync(slotName);

        if (!meta.exists)
        {
            m_TitleText.text = $"{slotName} (Empty)";
            m_ThumbnailImage.gameObject.SetActive(false);
            return;
        }

        m_TitleText.text = string.IsNullOrEmpty(meta.displayName) ? meta.slotName :
meta.displayName;
        m_PlaytimeText.text = $"Playtime: {meta.GetFormattedPlaytime()}";
        m_DateText.text = $"Saved: {meta.GetFormattedDate()}";

        // Restore Thumbnail from Base64
        if (!string.IsNullOrEmpty(meta.thumbnailBase64))
        {
            byte[] imageBytes = Convert.FromBase64String(meta.thumbnailBase64);
            Texture2D tex = new Texture2D(2, 2);
            if (tex.LoadImage(imageBytes))
            {
                m_ThumbnailImage.texture = tex;
                m_ThumbnailImage.gameObject.SetActive(true);
            }
        }
    }
}
```

```
// Display Summary Stats
if (meta.summaryValues.TryGetValue("level", out string level))
{
    m_SummaryText.text = $"Level: {level}";
}
}
}
```

Next Chapter

Proceed to [09. Cloud Synchronization](#) to explore multi-provider cloud saves and conflict resolution strategies.

09. Cloud Synchronization Overview

In cross-platform and live-service gaming, players expect their progress to sync across devices (e.g., PC Steam, mobile devices, and consoles).

FuzzySave features a provider-agnostic **Cloud Synchronization Architecture** that decouples game logic from specific cloud backends.

Architecture: ICloudStorageProvider

All cloud providers implement a unified interface:

```
public interface ICloudStorageProvider
{
    bool IsAuthenticated { get; }
    Task<bool> AuthenticateAsync();
    Task<bool> UploadAsync(string slotName, byte[] payload);
    Task<byte[]> DownloadAsync(string slotName);
    Task<bool> IsCloudNewerAsync(string slotName, long localTimestamp);
}
```

Sync Lifecycle States (CloudSyncState):

CloudSyncManager.OnSyncStateChanged notifies your game UI of active sync states:

- Idle: No active cloud network operations.
 - Uploading: Sending save bytes to the remote server.
 - Downloading: Fetching latest cloud data.
 - Conflict: Cloud and local timestamps diverge, requiring resolution.
 - Error: Network timeout, authentication failure, or invalid payload.
-

Conflict Resolution Policies

When a player plays offline on their laptop and then switches to their desktop, local and remote save versions may conflict:

Conflict Policy	Behavior	Best Used For
UseNewest (Default)	Compares UTC timestamp ticks of local and cloud saves; the newer save wins.	Casual & story-driven single-player games.
AlwaysLocal	Local save always overwrites the remote file upon upload.	Offline-first mobile games with local priority.
AlwaysCloud	Cloud save always takes precedence if remote changes exist.	Competitive or cheat-sensitive titles with server authority.

Supported Cloud Providers

1. Unity Gaming Services (UGS) Cloud Save

- **Class:** `UGSCloudStorageProvider`
- Integrates directly with Unity's official Cloud Save package (`#if UNITY_SERVICES_CORE && UNITY_SERVICES_CLOUDSAVE`).
- Saves binary or JSON payloads directly to the authenticated player's cloud profile.

2. Steam Cloud (Steamworks)

- **Class:** `SteamCloudStorageProvider`
- Compatible with both **Steamworks.NET** and **Facepunch.Steamworks**.
- Uses Steam's Remote Storage API (`SteamRemoteStorage.FileWrite` and `SteamRemoteStorage.FileRead`).
- Seamlessly synchronizes save files across different Steam client installations without requiring custom web infrastructure.

3. Microsoft Azure PlayFab

- **Class:** PlayFabCloudStorageProvider
- Interfaces with PlayFab Client API user data entries (UpdateUserData / GetUserData).
- Excellent for cross-platform progression between PC, iOS, and Android.

4. Google Firebase

- **Class:** FirebaseCloudStorageProvider
- Uploads compressed and encrypted save payloads to Firebase Storage or Realtime Database.

5. Custom REST API

- **Class:** RESTCloudStorageProvider
 - Built for studios with custom backend web servers using UnityWebRequest:
 - **Upload:** POST {restEndpointUrl}/upload?slot={slotName}
 - **Download:** GET {restEndpointUrl}/download?slot={slotName}
 - **Metadata Check:** GET {restEndpointUrl}/metadata?slot={slotName}
-

Manual Cloud Sync Code Example

While cloud sync triggers automatically after local saves when enabled in FuzzySaveSettings, you can also trigger synchronization manually:

```
using UnityEngine;
using FuzzyLogicLabs.FuzzySave;
using System.Threading.Tasks;

public class CloudSaveController : MonoBehaviour
{
    private async void Start()
    {
        // Listen to sync events
        CloudSyncManager.OnSyncStateChanged += HandleSyncStateChanged;

        // Ensure cloud provider is authenticated
        if (CloudSyncManager.Provider != null &&
            !CloudSyncManager.Provider.IsAuthenticated)
        {
            bool authed = await CloudSyncManager.Provider.AuthenticateAsync();
            Debug.Log($"Cloud Authenticated: {authed}");
        }
    }

    private void HandleSyncStateChanged(CloudSyncState state, string slot)
    {
        Debug.Log($"[CloudSync] Slot: {slot} -> Status: {state}");
    }
}
```

Next Chapter

Proceed to [10. Schema Migrations & Versioning](#) to learn how to update game data structures without breaking older player save files.

10. Schema Migrations & Versioning Overview

As your game evolves through updates, DLCs, and patches, your data models inevitably change:

- A field is renamed (e.g., gold becomes coins).
- A new mechanic is introduced requiring default stats (e.g., mana or skill_points).
- An obsolete data structure is deprecated or refactored.

Without schema migration, loading an older save file into a patched game leads to missing data, null reference exceptions, or corrupted game states.

FuzzySave includes a dedicated **Chained Save Migration Engine** (SaveMigrationManager) to ensure backwards compatibility across all game versions.

Chained Migration Architecture

Every save file records the saveVersion under which it was created. When LoadAsync is called, FuzzySave checks if the file's version matches FuzzySaveSettings.currentSaveVersion.

If the file is older, migrations run sequentially in a chain:

```
flowchart LR
    File["Save File (v1)"] --> Step1["Migration v1 → v2<br/>(Rename 'gold' to 'coins')"]
    Step1 --> Step2["Migration v2 → v3<br/>(Inject 'difficulty' = 'Normal')"]
    Step2 --> Restored["Target Engine (v3 Active)"]
```

Registering Migrations

Register migrations during game startup before any scenes are loaded using Unity's [RuntimeInitializeOnLoadMethod]:

```

using UnityEngine;
using FuzzyLogicLabs.FuzzySave;

public static class GameSaveMigrations
{
    [RuntimeInitializeOnLoadMethod(RuntimeInitializeLoadType.BeforeSceneLoad)]
    public static void InitializeMigrations()
    {
        // -----
        // Step 1: Migrate Version 1 -> Version 2
        // Problem: 'gold' field renamed to 'coins'
        // -----
        SaveMigrationManager.RegisterMigration(fromVersion: 1, toVersion: 2, container
=>
    {
        var goldEntry = container.entries.Find(e => e.key == "gold");
        if (goldEntry != null)
        {
            // Create new entry with migrated key
            container.entries.Add(new SaveEntry
            {
                key = "coins",
                typeName = goldEntry.typeName,
                jsonValue = goldEntry.jsonValue
            });

            // Remove deprecated key
            container.entries.Remove(goldEntry);
            Debug.Log("[FuzzySave Migration] Migrated 'gold' -> 'coins'
successfully.");
        }
    });

    // -----
    // Step 2: Migrate Version 2 -> Version 3
    // Problem: Added 'difficulty' setting with default "Normal"
    // -----
    SaveMigrationManager.RegisterMigration(fromVersion: 2, toVersion: 3, container
=>
    {
        if (!container.entries.Exists(e => e.key == "difficulty"))
        {

```

```
        container.entries.Add(new SaveEntry
        {
            key = "difficulty",
            typeName = typeof(string).AssemblyQualifiedName,
            jsonValue = "\"Normal\"" // JSON string literal
        });
        Debug.Log("[FuzzySave Migration] Injected default 'difficulty'
setting.");
    }
});
}
```

Best Practices for Versioning

1. **Increment currentSaveVersion in FuzzySaveSettings:** Whenever you make breaking changes to saved variables, increment the version number in your settings asset.
2. **Always Keep Old Migrations in Code:** Never delete older migration steps (e.g., v1 -> v2). A player might return to your game after two years of updates and need to jump directly from v1 to v4!
3. **Automate via Tests:** Create unit tests that feed sample v1 JSON containers into SaveMigrationManager to verify that the migrated output matches expected modern schemas.

Next Chapter

Proceed to [11. Attributes & Code-First Development](#) to explore C# attribute decorators for granular code-first persistence.

11. Attributes & Code-First Development Overview

While Visual Save Studio provides a visual workflow for technical designers, programmers often prefer **Code-First Architecture**.

FuzzySave provides declarative C# attributes that allow developers to tag classes, properties, fields, and events directly in script files.

Attribute Reference Table

Attribute	Valid Targets	Purpose
[FuzzySave]	Class, Struct	Registers the type with FuzzySave. Accepts an optional default Save Group name.
[SaveField]	Field, Property	Explicitly includes a variable in the save container. Accepts an optional key alias.
[SaveGroup]	Class, Field	Binds a class or field to a specific group, with optional encryption and compression flags.
[SaveEvent]	Event, Action	Automatically triggers an asynchronous save of the target group whenever this event fires.
[SaveExclude]	Field, Property	Explicitly excludes a public or serialized field from being saved.

Code-First Example: Player Character & Events

```
using System;
using System.Collections.Generic;
using UnityEngine;
using FuzzyLogicLabs.FuzzySave;

[FuzzySave("PlayerGroup")]
public class PlayerHero : MonoBehaviour
{
    // Custom key alias saves disk space in JSON
    [SaveField("hp")]
    public int currentHealth = 100;

    [SaveField("mana")]
    public float currentMana = 50.0f;

    [SaveField("coins")]
    private int m_GoldCoins = 250;

    // Complex types are fully supported
    [SaveField("inventory")]
    public List<string> equippedGear = new List<string> { "Iron_Sword",
"Leather_Boots" };

    // Explicitly ignored
    [SaveExclude]
    public float temporarySpeedBoost = 1.0f;

    // Event-Driven Auto Save:
    // Whenever OnLevelCompleted is invoked, "PlayerGroup" is automatically saved!
    [SaveEvent("PlayerGroup")]
    public event Action OnLevelCompleted;

    public void FinishLevel()
    {
        currentHealth = 100;
        m_GoldCoins += 500;

        // Invoking this automatically triggers a background save of PlayerGroup!
        OnLevelCompleted?.Invoke();
    }
}
```

```
}
```

Group-Specific Encryption via Attributes

You can isolate sensitive game values (like in-game currency or microtransaction unlocks) into an encrypted group while leaving regular gameplay settings unencrypted:

```
[SaveGroup("SecuritySensitive", Encrypted = true, Compressed = true)]
public class PlayerWallet : MonoBehaviour
{
    [SaveField("premium_gems")]
    public int premiumGems = 100;

    [SaveField("vip_status")]
    public bool isVip = true;
}
```

Seamless Harmony with Visual Save Studio

Attributes and Visual Save Studio are not mutually exclusive:

- If a class is decorated with `[FuzzySave]`, Visual Save Studio's discovery scanner recognizes the attributes and flags the fields automatically.
- Any manual overrides configured in Visual Save Studio seamlessly take precedence when code is baked.

Next Chapter

Proceed to [12. Samples & Stress Benchmark Suite](#) to explore the interactive 3D demo world and 500+ physics item stress test suite.

DTOs & Zero-Allocation Serialization

1. Why DTOs (Data Transfer Objects)?

In standard Unity serialization pipelines, developers often attempt to serialize native Unity structs (Vector3, Quaternion, Color) directly with standard JSON libraries. This introduces multiple performance and stability penalties:

- 1. **Garbage Collection (GC) Overhead:** Serializers relying on reflection allocate boxed objects on the managed heap, triggering garbage collection hitches.
- 2. **Verbose JSON Payload:** Native Unity types frequently contain internal properties (normalized, magnitude, sqrMagnitude, eulerAngles) that bloat the serialized file by up to 400%.
- 3. **Circular Reference Risks:** Complex Unity objects can trigger infinite recursion exceptions in third-party serializers.

FuzzySave addresses this with a dedicated library of lightweight, serializable structs located in `DTOs.cs`.

2. The DTO Struct Suite

Every DTO is declared as a `[Serializable]` C# struct (value type) containing only primitive numerical fields.

2.1 Supported Mathematical & Geometric DTOs

DTO Type	Corresponding Unity Type	Underlying Fields	Implicit Casting
Vector2DTO	UnityEngine.Vector2	float x, y	Yes (Bidirectional)
Vector3DTO	UnityEngine.Vector3	float x, y, z	Yes (Bidirectional)
Vector4DTO	UnityEngine.Vector4	float x, y, z, w	Yes (Bidirectional)

DTO Type	Corresponding Unity Type	Underlying Fields	Implicit Casting
QuaternionDTO	UnityEngine.Quaternion	float x, y, z, w	Yes (Bidirectional)
ColorDTO	UnityEngine.Color	float r, g, b, a	Yes (Bidirectional)
Color32DTO	UnityEngine.Color32	byte r, g, b, a	Yes (Bidirectional)
RectDTO	UnityEngine.Rect	float x, y, width, height	Yes (Bidirectional)
BoundsDTO	UnityEngine.Bounds	Vector3DTO center, size	Yes (Bidirectional)

3. Seamless Implicit Conversions

All FuzzySave DTO structs implement C# **implicit operator conversions**. Developers can assign native Unity types directly to DTO fields—and vice versa—without explicit cast syntax or memory allocation:

```
// Assigning native Vector3 directly to Vector3DTO
Vector3DTO savedPos = transform.position;

// Assigning Vector3DTO directly back to native Vector3
Vector3 livePos = savedPos;

// Same seamless syntax for Colors and Quaternions
ColorDTO playerColor = Color.cyan;
Color nativeColor = playerColor;
```

4. TransformDTO & Physics Controller Safeguards

Persisting a GameObject's complete transform state requires tracking position, rotation, scale, and active state simultaneously. TransformDTO wraps these into a single compact structure:

```
[Serializable]
public struct TransformDTO
{
    public Vector3DTO position;
    public QuaternionDTO rotation;
    public Vector3DTO localScale;
    public bool activeSelf;
}
```

The CharacterController & NavMeshAgent Teleport Problem

In Unity, directly modifying `transform.position` on a GameObject with an active CharacterController or NavMeshAgent is notoriously error-prone:

- The CharacterController will often reject the position change or cause rubber-banding physics glitches.
- The NavMeshAgent will log errors regarding position mismatch or attempt to pathfind backwards to the previous coordinate.

`TransformDTO.ApplyTo` solves this automatically:

```

public void ApplyTo(Transform transform, SaveFlags flags = SaveFlags.All)
{
    if (transform == null) return;

    if (flags.HasFlag(SaveFlags.Position))
    {
        var cc = transform.GetComponent<UnityEngine.CharacterController>();
        var agent = transform.GetComponent<UnityEngine.AI.NavMeshAgent>();

        // 1. Temporarily disable physics/navmesh controllers
        if (cc != null) cc.enabled = false;
        if (agent != null) agent.enabled = false;

        // 2. Safely apply position matrix
        transform.position = position;

        // 3. Re-enable controllers cleanly at the new destination
        if (cc != null) cc.enabled = true;
        if (agent != null) agent.enabled = true;
    }

    if (flags.HasFlag(SaveFlags.Rotation))
        transform.rotation = rotation;

    if (flags.HasFlag(SaveFlags.Scale))
        transform.localScale = localScale;

    if (flags.HasFlag(SaveFlags.ActiveState))
        transform.gameObject.SetActive(activeSelf);
}

```

This guarantees that character controllers, AI agents, and physics objects can be loaded into new coordinates reliably.

12. Samples & Stress Benchmark Suite

Overview

FuzzySave ships with two production-ready test scenes:

1. **Interactive 3D Demo Scene (DemoScene.unity):** A polished, stylized third-person environment showcasing player controls, interactive chests, dynamic loot spawning, and UI slot cards.
 2. **Performance & Stress Benchmark Arena (PerformanceBenchmarkScene.unity):** An extreme stress-testing environment capable of spawning and verifying 500+ physics items, 100+ AI agents, graveyard destruction tracking, and automated golden baseline audits.
-

1. Interactive 3D Demo Scene

Located in: Assets/FuzzyLogicLabs/FuzzySave/Samples/Demo/DemoScene.unity

Gameplay Components:

- **DemoPlayerController:** Character movement built on CharacterController with smooth camera follow, footsteps, and damage/heal effects. Decorated with `[FuzzySave("PlayerGroup")]`.
- **DemoChest:** Smoothly opens using `Mathf.SmoothStep`. Features hybrid raycasting input that seamlessly supports both Unity's **Legacy Input Manager** and the **New Input System** package. Monitored via SaveGuid.
- **DemoLootSpawner & DemoFloatingItem:** Press **L** or click the UI button to spawn rotating crystal loot. Each crystal uses `DynamicSpawnTracker` to record its world position, bobbing speed, and rotational velocity. Upon loading, all scattered crystals respawn in their exact positions.
- **DemoDataTypes:** A validation component demonstrating full persistence across:
 - Primitives (`int`, `float`, `bool`, `string`)
 - Unity Types (`Vector3D0`, `ColorD0`)
 - Collections (`List<int>`, `Dictionary<string, float>`)
 - Nested Custom Classes (`CustomWeaponData`)

- **DemoUIManager:** Dynamic HUD displaying player health bars, gold count, toast popups (DemoToastNotification), and 3 active slot buttons that update their date and file status dynamically.

2. Performance & Stress Benchmark Arena

Located in: Assets/FuzzyLogicLabs/FuzzySave/Samples/PerformanceBenchmark/

Automatic Arena Builder (PerformanceSceneBuilder.cs)

You do not need to manually configure the benchmark scene. Open: **Tools > FuzzySave > Build Performance Benchmark Scene**

The builder automatically:

1. Generates stylized materials and lighting.
2. Constructs the arena walls, floor, and drop origin.
3. Instantiates 100 roaming BenchmarkAgent entities.
4. Generates monument structures with SaveGuid components.
5. Builds the physics prefab in Resources/ and registers it inside FuzzySaveSettings.registeredPrefabs.
6. Configures the complete UI canvas and binds all stress test buttons.

Benchmark Stress Testing Controls

UI Button	Action Executed	What It Tests
Spawn 50 / 250 / 500	Spawns dynamic physics cubes into the arena.	DynamicSpawnTracker throughput and time-sliced instantiation.
Wipe Spawns	Destroys all spawned physics items.	Dynamic object cleanup and garbage collection stability.
Destroy 5 / Destroy All	Destroys monument props in the scene.	GraveyardRegistry tracking and scene object persistence.

UI Button	Action Executed	What It Tests
Revive Graveyard	Clears destruction records and revives props.	Real-time object revival and save container synchronization.
Randomize / Scramble Agents	Randomly mutates agent stats and positions.	Bulk memory state gathering across 100+ entities.
Run Audit	Audits live state against saved baseline.	100.0% Cryptographic Data Fidelity Verification.

The Golden Baseline Audit System

To prove mathematical correctness and data integrity, StressBenchmarkManager includes an automated **Golden Baseline Audit**:

```
[AUDIT VERIFICATION REPORT]
1. Dynamic Spawns Count: Expected 500, Restored 500. [PASS]
2. Graveyard Dead Entities: Expected 12, Suppressed 12. [PASS]
3. Agent Position Tolerances: All within 0.001f deviation. [PASS]
4. Custom Data Variables: 100% hash match. [PASS]
OVERALL STATUS: AUDIT PASSED (0 ERRORS DETECTED)
```

- 1. **Before Saving:** FuzzySave records a deep mathematical snapshot of every entity’s position, velocity, variable hash, and graveyard status.
- 2. **After Loading:** It scans the entire restored scene and validates every single object against the baseline.
- 3. If an object is missing, floating, wrongly oriented, or if a destroyed entity leaked through, the audit alerts the developer immediately.

Next Chapter

Proceed to [13. Complete C# API Reference](#) for the definitive programmatic reference for FuzzySaveManager.

13. Complete C# API Reference

Overview

The static `FuzzySaveManager` class serves as the primary gateway to all persistence, snapshotting, and slot management routines.

All methods are thread-safe or automatically marshal execution to the appropriate `Unity SynchronizationContext`.

1. In-Memory Data Access

`SetData<T>`

Stores a strongly-typed value in the current in-memory save container.

```
public static void SetData<T>(string key, T value)
```

- **Parameters:**

- `key (string)`: Unique identifier for this data entry.
- `value (T)`: The value to store (primitives, structs, classes, or collections).

- **Example:**

```
FuzzySaveManager.SetData("player_score", 4500);  
FuzzySaveManager.SetData("checkpoint_pos", (Vector3D0)transform.position);
```

`GetData<T>`

Retrieves a strongly-typed value from the current in-memory save container.

```
public static T GetData<T>(string key, T defaultValue = default)
```

- **Parameters:**

- `key (string)`: The identifier of the data entry to retrieve.
- `defaultValue (T)`: Fallback value returned if the key does not exist.

- **Returns:** The deserialized value or `defaultValue`.

- **Example:**

```
int score = FuzzySaveManager.GetData<int>("player_score", defaultValue: 0);
```

SetRawJson

Directly stores a pre-serialized JSON string without double-serializing.

```
public static void SetRawJson(string key, string json)
```

HasData & DeleteData

```
public static bool HasData(string key)
public static void DeleteData(string key)
```

- **Example:**

```
if (FuzzySaveManager.HasData("quest_boss_slain"))
{
    FuzzySaveManager.DeleteData("quest_boss_slain");
}
```

2. Asynchronous Save & Load Pipeline

SaveAsync

Asynchronously serializes and writes the current game state to local disk (and triggers cloud upload if enabled).

```
public static Task<bool> SaveAsync(string slotName = null)
```

- **Parameters:**

- slotName (string, optional): Target slot filename. If null or omitted, saves to FuzzySaveManager.ActiveSlot.

- **Returns:** Task<bool> resolving to true if save succeeded; otherwise false.

- **Example:**

```
bool saved = await FuzzySaveManager.SaveAsync("slot_campaign_1");
```

LoadAsync

Asynchronously reads and restores the game state from disk, reconstructing entities, applying variables, and handling cross-scene transitions if necessary.

```
public static Task<bool> LoadAsync(string slotName = null)
```

- **Parameters:**

- slotName (string, optional): Target slot filename to read. Defaults to ActiveSlot.

- **Returns:** Task<bool> resolving to true if load and restoration succeeded.

- **Example:**

```
bool loaded = await FuzzySaveManager.LoadAsync("slot_campaign_1");
```

SaveGroupAsync & LoadGroupAsync

Selectively saves or loads a specific Save Group instead of serializing the entire scene.

```
public static Task<bool> SaveGroupAsync(string groupName, string slotName = null)
public static Task<bool> LoadGroupAsync(string groupName, string slotName = null)
```

- **Example:**

```
// Save only inventory items without saving enemy or world positions:
await FuzzySaveManager.SaveGroupAsync("InventoryGroup");
```

3. RAM Snapshots (Time-Travel)

TakeSnapshot

Captures the entire active memory container into high-speed RAM.

```
public static void TakeSnapshot(string snapshotId)
```

RestoreSnapshot

Restores game state from an in-memory snapshot within < 1ms without disk I/O.

```
public static void RestoreSnapshot(string snapshotId)
```

- **Example:**

```
// Before boss encounter:  
FuzzySaveManager.TakeSnapshot("boss_room_checkpoint");  
  
// On player death:  
FuzzySaveManager.RestoreSnapshot("boss_room_checkpoint");
```

4. Slot & Companion Metadata

ActiveSlot

Property to get or set the default active slot filename.

```
public static string ActiveSlot { get; set; } // Default: "DefaultSlot"
```

SlotExists & DeleteSlot

```
public static bool SlotExists(string slotName = null)  
public static void DeleteSlot(string slotName = null)  
public static void ClearAllSlots()  
public static string[] GetAvailableSlots()
```

GetSlotMetadataAsync & GetAllSlotMetadataAsync

Reads companion .meta files without loading multi-megabyte save payloads.

```
public static Task<SaveSlotMetadata> GetSlotMetadataAsync(string slotName = null)
public static Task<List<SaveSlotMetadata>> GetAllSlotMetadataAsync()
```

SetSlotSummaryValue & GetSlotSummaryValue

```
public static void SetSlotSummaryValue(string key, string value)
public static string GetSlotSummaryValue(string key)
```

- **Example:**

```
FuzzySaveManager.SetSlotSummaryValue("difficulty", "Hardcore");
FuzzySaveManager.SetSlotSummaryValue("level", "45");
```

5. GPU Thumbnail Screen Capture

CaptureThumbnailBase64Async

Asynchronously captures the current frame buffer via AsyncGPUReadback and encodes it to a Base64 JPEG string.

```
public static Task<string> CaptureThumbnailBase64Async(int width = 240, int height
= 135, int quality = 70)
```

6. Graveyard & Destruction Tracking

```
// Destroy GameObject and register GUID to prevent respawning on reload:
public static void Destroy(GameObject go, float t = 0f)
public static void Destroy(SaveGuid saveGuid)

// Query if a GUID is destroyed:
public static bool IsDestroyed(string guid)

// Revive a destroyed entity:
public static void Revive(string guid)

// Clear all graveyard records:
public static void ClearGraveyard()
public static List<string> GetDestroyedGuids()
```

7. Diagnostics & Profiling Properties

```
// Duration of Main Thread gathering during last save (ms)
public static float LastSaveMainThreadGatherMs { get; }

// Duration of Background Worker Thread I/O during last save (ms)
public static float LastSaveWorkerDurationMs { get; }

// State flags
public static bool IsSaving { get; }
public static bool IsLoading { get; }
public static bool IsBusy { get; }

// Total cumulative playtime in seconds
public static float TotalPlaytimeSeconds { get; }
```

8. Global Events & Delegates

```
public static event Action<string> OnSaveStarted;
public static event Action<string, bool> OnSaveCompleted;
public static event Action<string> OnLoadStarted;
public static event Action<string, bool> OnLoadCompleted;
public static event Action<string> OnActiveSlotChanged;
public static event Action<string> OnCustomEventTriggered;

// Manually fire a custom event rule:
public static void TriggerEvent(string eventName)
```